

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Операционные системы»

Выполнил: В. А. Яшнов
Группа: М8О-203Б-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: изучение системных вызовов операционных систем Windows и Linux для создания процессов и анонимных каналов, а также приобретение навыков построения кроссплатформенной обёртки (единого API) над системными примитивами.

Задание: разработать программу, состоящую из двух процессов:

- родительский процесс `parent` считывает со стандартного ввода строки, содержащие числа с плавающей запятой, и передаёт их дочернему процессу по первому анонимному каналу;
- дочерний процесс `child` получает числа со стандартного ввода, накапливает их сумму, записывает отчёт в указанный в аргументе файл и возвращает итоговую сумму родителю по второму анонимному каналу;
- родительский процесс выводит полученную сумму и ожидает завершения дочернего процесса.

Программа должна собираться и выполняться одинаково в средах Windows и Linux: логика приложений не должна содержать системных вызовов, всё платформозависимое должно быть скрыто за единым кроссплатформенным API, выбор реализации (Win32 или POSIX) должен выполняться на этапе сборки проекта (CMake).

Вариант: 2

Метод решения

Логика программы разделена на три независимых уровня:

1. *общий интерфейс* `src/os_api.h` — абстракции `os_pipe` и `os_process`, а также единый набор функций для создания и использования каналов и процессов;
2. *платформозависимые реализации* `src/os_api_win.c` (Win32 API) и `src/os_api_posix.c` (POSIX API) — здесь находятся все системные вызовы, условная компиляция отсутствует, выбор файла выполняется в `CMakeLists.txt`;
3. *прикладной уровень* `src/app/parent.c` и `src/app/child.c` — вызывают только функции единого API, никаких системных вызовов в коде приложений нет.

Алгоритм работы родительского процесса:

1. создаются два канала: `in` (родитель → ребёнок) и `out` (ребёнок → родитель);
2. через `os_process_spawn` запускается `child` с именем файла-отчёта; обёртка самостоятельно подключает каналы к стандартному вводу/выводу дочернего процесса и закрывает ненужные в текущем процессе концы каналов;
3. в цикле читаются строки со стандартного ввода, числа с плавающей запятой извлекаются и записываются в канал `in`;
4. по окончании ввода закрывается записывающий конец канала `in` — это сигнал EOF для дочернего процесса;

5. родитель читает итоговую сумму из канала `out`, закрывает каналы, ожидает завершения `child(os_process_wait)` и закрывает дескриптор процесса.

Дочерний процесс получает числа через стандартный ввод, суммирует их, записывает строку отчёта в заданный файл и передаёт сумму обратно через стандартный вывод. Выбор низкоуровневых деталей (преобразование `argv` в командную строку Windows, добавление `.exe`, префикс `./` на POSIX и т. п.) полностью инкапсулирован в реализации обёртки.

Описание программы

Разделение по файлам:

<code>src/os_api.h</code>	единый интерфейс (типы, объявления функций)
<code>src/os_api_win.c</code>	реализация для Windows (Win32 API)
<code>src/os_api_posix.c</code>	реализация для Linux (POSIX API)
<code>src/app/parent.c</code>	родительский процесс
<code>src/app/child.c</code>	дочерний процесс
<code>CMakeLists.txt</code>	выбор реализации, сборка обоих исполняемых файлов

Основные типы данных:

- `os_pipe` — два дескриптора (`read`, `write`) типа `uintptr_t`; значение `OS_INVALID` означает закрытый/несозданный дескриптор;
- `os_process` — дескриптор процесса (`proc`) и его идентификатор (`pid`, используется для логирования).

Функции единого API и их семантика:

<code>os_self_pid()</code>	PID текущего процесса
<code>os_pipe_create()</code>	создание анонимного канала
<code>os_pipe_close_read/write()</code>	закрытие конца канала
<code>os_pipe_read()/write()</code>	чтение/запись данных через канал
<code>os_process_spawn()</code>	запуск дочернего процесса с перенаправлением каналов
<code>os_process_pid()</code>	PID запущенного процесса
<code>os_process_wait()</code>	ожидание завершения процесса и получение кода выхода
<code>os_process_close()</code>	освобождение ресурсов процесса

Все функции возвращают 0 при успехе и -1 при ошибке (исключение — `os_pipe_read`: 1 — прочитано ровно `size` байт, 0 — EOF).

Используемые системные вызовы:

POSIX (Linux)

`pipe`, `getpid`, `read`,
`write`, `close`

`fork`, `dup2`, `execv`,
`waitpid`

Win32 (Windows)

`CreatePipe`,
`GetCurrentProcessId`, `ReadFile`,
`WriteFile`, `CloseHandle`,
`SetHandleInformation`,
`CreateProcessW`,
`WaitForSingleObject`,
`GetExitCodeProcess`

Обработка ошибок: каждая платформозависимая функция при сбое формирует диагностическое сообщение `report()`: на POSIX используются `errno` и `strerror()`, на Windows — `GetLastError()` и `FormatMessageA()`. Приложение анализирует коды возврата функций API: любая ошибка приводит к коду возврата процесса 1, при этом ресурсы (каналы и дескриптор процесса) освобождаются всегда. Для предотвращения гибели родительского процесса от сигнала `SIGPIPE` при записи в уже закрытый канал в POSIX-реализации сигнал игнорируется, а ошибка обрабатывается штатно через `EPIPE`.

Результаты

Программа успешно собрана и запущена в двух операционных системах — Windows и Linux — из одной и той же папки исходных текстов с единой логикой приложений. Реализация обёртки выбиралась автоматически на этапе конфигурации CMake.

Для проверки вводилась последовательность чисел. Работа подтверждается журналами процессов:

- `[parent pid=]` — отправленные числа и полученная сумма;
- `[child pid=]` — полученные числа, нарастающая сумма и факт записи файла-отчёта;
- финальный код выхода обоих процессов — 0.

```
PS C:\Users\Vlad\Desktop\Учебное\Предметы\Операционные системы\os_labs\lab_2\build-windows\bin\Debug> ./parent.exe
Enter file name for child: test_windows
[parent pid=10112] child started, pid=12460
Enter any number of float numbers per line, space-separated.
Empty line or EOF to finish.
> [child pid=12460] started, file='test_windows'
1 2.01 16
[parent pid=10112] sent 1.000
[child pid=12460] got 1.000, running sum = 1.000
[parent pid=10112] sent 2.010
[child pid=12460] got 2.010, running sum = 3.010
[parent pid=10112] sent 16.000
[child pid=12460] got 16.000, running sum = 19.010
[parent pid=10112] line done: 3 number(s)
> 10
[parent pid=10112] sent 10.000
[parent pid=10112] line done: 1 number(s)
> [child pid=12460] got 10.000, running sum = 29.010

[parent] pipe1 closed, waiting for child's reply...
[child pid=12460] EOF on stdin, 4 number(s) total
[child pid=12460] result written to 'test_windows'
[child pid=12460] exiting
[parent pid=10112] child reported sum = 29.010
[parent] child exited with code 0
```

Рис. 1: Запуск `parent` в Windows

Ключевые особенности полученного решения:

- кроссплатформенность: код приложений не содержит системных вызовов и полностью идентичен для обеих ОС;
- корректная передача EOF: закрытие записывающего конца канала родителем завершает чтение дочернего процесса;
- обработка ошибок: при любой системной ошибке оба процесса завершаются с ненулевым кодом, ресурсы освобождаются; на Linux предотвращена гибель родителя по `SIGPIPE` при записи в закрытый канал.

```

vscode → ../os_labs/lab_2/build-linux/bin $ ./parent
Enter file name for child: test_linux
[parent pid=700] child started, pid=729
Enter any number of float numbers per line, space-separated.
Empty line or EOF to finish.
> [child pid=729] started, file='test_linux'
1 2 3 4 5
[parent pid=700] sent 1.000
[parent pid=700] sent 2.000
[parent pid=700] sent 3.000
[parent pid=700] sent 4.000
[parent pid=700] sent 5.000
[parent pid=700] line done: 5 number(s)
> [child pid=729] got 1.000, running sum = 1.000
[child pid=729] got 2.000, running sum = 3.000
[child pid=729] got 3.000, running sum = 6.000
[child pid=729] got 4.000, running sum = 10.000
[child pid=729] got 5.000, running sum = 15.000
0.0001 0
[parent pid=700] sent 0.000
[parent pid=700] sent 0.000
[parent pid=700] line done: 2 number(s)
[child pid=729] got 0.000, running sum = 15.000
> [child pid=729] got 0.000, running sum = 15.000

[parent] pipe1 closed, waiting for child's reply...
[child pid=729] EOF on stdin, 7 number(s) total
[child pid=729] result written to 'test_linux'
[child pid=729] exiting
[parent pid=700] child reported sum = 15.000
[parent] child exited with code 0

```

Рис. 2: Запуск parent в Linux

Выводы

В ходе выполнения лабораторной работы были изучены и использованы конкретные системные вызовы операционных систем Windows (Win32 API) и Linux (POSIX) для работы с процессами и анонимными каналами: `CreatePipe`, `CreateProcessW`, `WaitForSingleObject`, `pipe`, `fork`, `dup2`, `execv`, `waitpid`.

Разработана программа из двух взаимодействующих через пайпы процессов в соответствии с ТЗ. Программа выполнена на языке C по схеме «единый API + две платформозависимые реализации», благодаря чему логика приложений полностью кроссплатформенна, а выбор реализации выполняется на этапе сборки.

Реализовано детализированное логирование всех процессов и системных вызовов. Исправлены ошибки организации проекта, допущенные при старте работы. Отлажены краевые случаи падения процессов с появлением фантомных исходов или полным их отсутствием. Достигнута равная работоспособность на Windows и Linux из единой исходной базы. Подтверждена корректная обработка ошибок и завершение обоих процессов с ненулевым кодом при сбоях. Сформирован полноценный отчет о ходе и результатах проделанной работы.

Исходная программа

Сборка и запуск:

- Windows: `cmake -S . -B build-windows && cmake --build build-windows`, запуск `./build-windows/bin/Release/parent.exe` из папки с `child.exe`;
- Linux: `cmake -S . -B build-linux && cmake --build build-linux`, запуск `./build-linux/bin/parent` из папки с `child`;
- файл с исходными текстами содержит папку `src` (в том числе `src/app`) и корневой `CMakeLists.txt`.

Единый интерфейс:

```
1  #ifndef LAB2_OS_API_H
2  #define LAB2_OS_API_H
3
4  #include <stddef.h>
5  #include <stdint.h>
6
7  typedef struct {
8      uintptr_t read; // int or HANDLE
9      uintptr_t write;
10 } os_pipe;
11
12 typedef struct {
13     uintptr_t proc; // PID or HANDLE
14     unsigned long pid; // for logs
15 } os_process;
16
17 unsigned long os_self_pid(void); // PID of current process
18
19 int os_pipe_create(os_pipe *p); // 0 - success; -1 - error (reason to
    stderr)
20
21 int os_pipe_close_read(os_pipe *p); // 0 - success; -1 - error (reason to
    stderr)
22 int os_pipe_close_write(os_pipe *p); // 0 - success; -1 - error (reason to
    stderr)
23
24 int os_pipe_read(os_pipe *p, void *buf, size_t size); // 1 - read size
    bytes to buf; 0 - got EOF; -1 - error
25 int os_pipe_write(os_pipe *p, const void *buf, size_t size); // 0 - wrote
    size bytes from buf; -1 - error
26
27 int os_process_spawn(os_process *out, const char *const argv[],
    os_pipe *in, os_pipe *out_pipe);
28
29 /*
30     argv[0] - executable file name,
31     argv[1...n-1] - args for executable
32     argv[n] - NULL
33     0 - success, -1 - error.
34 */
35
36 unsigned long os_process_pid(const os_process *p);
37
```

```

38 | int os_process_wait(os_process *p, int *exit_code);
39 | /*
40 |     wait until process ends;
41 |     0 - success, -1 - error
42 | */
43 |
44 | int os_process_close(os_process *p); // clear finished process resources; 0
    |     - success, -1 - error
45 |
46 | #endif

```

Листинг 1: src/os_api.h

Реализация для Windows:

```

1 | #include "os_api.h"
2 |
3 | #include <stdio.h>
4 | #include <stdlib.h>
5 | #include <string.h>
6 | #include <windows.h>
7 |
8 | #define OS_INVALID ((uintptr_t)-1) // == INVALID_HANDLE_VALUE
9 |
10 | static void report(const char *msg)
11 | {
12 |     DWORD err = GetLastError(); // DWORD - uint32_t
13 |     fprintf(stderr, "[os_api:%s] Win32 error %lu: ", msg, (unsigned long)err
    |         );
14 |
15 |     LPSTR buf = NULL;
16 |     FormatMessageA(
17 |         FORMAT_MESSAGE_ALLOCATE_BUFFER |
18 |         FORMAT_MESSAGE_FROM_SYSTEM |
19 |         FORMAT_MESSAGE_IGNORE_INSERTS,
20 |         NULL, err,
21 |         MAKELANGID(LANG_NEUTRAL, SUBLANG_DEFAULT),
22 |         (LPSTR)&buf, 0, NULL);
23 |     if (buf) {
24 |         fprintf(stderr, "%s", buf);
25 |         LocalFree(buf);
26 |     } else {
27 |         fprintf(stderr, "(no description)\n");
28 |     }
29 | }
30 |
31 | unsigned long os_self_pid(void)
32 | {
33 |     return (unsigned long)GetCurrentProcessId();
34 | }
35 |
36 | int os_pipe_create(os_pipe *p)
37 | {
38 |     SECURITY_ATTRIBUTES sa;
39 |     sa.nLength = sizeof(sa);
40 |     sa.lpSecurityDescriptor = NULL; // default

```

```

41     sa.bInheritHandle = TRUE;
42
43     HANDLE r = NULL, w = NULL;
44     if (!CreatePipe(&r, &w, &sa, 0)) {
45         report("CreatePipe");
46         return -1;
47     }
48     p->read = (uintptr_t)r;
49     p->write = (uintptr_t)w;
50     return 0;
51 }
52
53 int os_pipe_close_read(os_pipe *p)
54 {
55     if (p->read != OS_INVALID) {
56         if (!CloseHandle((HANDLE)p->read)) {
57             report("CloseHandle(read)");
58             return -1;
59         }
60         p->read = OS_INVALID;
61     }
62     return 0;
63 }
64
65 int os_pipe_close_write(os_pipe *p)
66 {
67     if (p->write != OS_INVALID) {
68         if (!CloseHandle((HANDLE)p->write)) {
69             report("CloseHandle(write)");
70             return -1;
71         }
72         p->write = OS_INVALID;
73     }
74     return 0;
75 }
76
77 static int set_inherit(HANDLE h, BOOL on)
78 {
79     return SetHandleInformation(h, HANDLE_FLAG_INHERIT,
80                                on ? HANDLE_FLAG_INHERIT : 0);
81 }
82
83 static int has_extension(const char *name)
84 {
85     const char *base = name;
86     for (const char *p = name; *p; ++p) {
87         if (*p == '/' || *p == '\\')
88             base = p + 1;
89     }
90     const char *dot = strrchr(base, '.');
91     return dot != NULL && dot != base && dot[1] != '\0';
92 }
93
94 static int build_cmdline(const char *const argv[], char *buf, size_t size)
95 {
96     size_t used = 0;
97     for (int i = 0; argv[i] != NULL; ++i) {
98         const char *s = argv[i];

```

```

99     size_t len = strlen(s);
100     size_t need = 2 /* кавычки */ + len + (i == 0 && !has_extension(s) ? 4
        /* ".exe" */ : 0);
101     if (i > 0)
102         need += 1; /* пробел */
103     if (used + need + 1 > size)
104         return -1;
105
106     if (i > 0)
107         buf[used++] = ' ';
108     buf[used++] = '"';
109     memcpy(buf + used, s, len);
110     used += len;
111     if (i == 0 && !has_extension(s)) {
112         memcpy(buf + used, ".exe", 4);
113         used += 4;
114     }
115     buf[used++] = '"';
116 }
117 buf[used] = '\\0';
118 return 0;
119 }
120
121 int os_pipe_read(os_pipe *p, void *buf, size_t size)
122 {
123     DWORD rd = 0;
124     if (!ReadFile((HANDLE)p->read, buf, (DWORD)size, &rd, NULL)) {
125         report("ReadFile");
126         return -1;
127     }
128     return rd == size ? 1 : 0;
129 }
130
131 int os_pipe_write(os_pipe *p, const void *buf, size_t size)
132 {
133     DWORD written = 0;
134     if (!WriteFile((HANDLE)p->write, buf, (DWORD)size, &written, NULL)) {
135         report("WriteFile");
136         return -1;
137     }
138     if (written != (DWORD)size) {
139         fprintf(stderr, "[os_api] short write\\n");
140         return -1;
141     }
142     return 0;
143 }
144
145 int os_process_spawn(os_process *out, const char *const argv[],
146                     os_pipe *in, os_pipe *out_pipe)
147 {
148     if (!set_inherit((HANDLE)in->read, TRUE)) {
149         report("SetHandleInformation(in.read)");
150         return -1;
151     }
152     if (!set_inherit((HANDLE)out_pipe->write, TRUE)) {
153         report("SetHandleInformation(out.write)");
154         return -1;
155     }

```

```

156     if (!set_inherit((HANDLE)in->write, FALSE)) {
157         report("SetHandleInformation(in.write)");
158         return -1;
159     }
160     if (!set_inherit((HANDLE)out_pipe->read, FALSE)) {
161         report("SetHandleInformation(out.read)");
162         return -1;
163     }
164
165     char *cmd = (char *)malloc(32768);
166     if (!cmd) {
167         fprintf(stderr, "[os_api] out of memory\n");
168         return -1;
169     }
170     if (build_cmdline(argv, cmd, 32768) != 0) {
171         fprintf(stderr, "[os_api] command line too long\n");
172         free(cmd);
173         return -1;
174     }
175
176     WCHAR wcmd[32768];
177     if (MultiByteToWideChar(CP_ACP, 0, cmd, -1, wcmd, 32768) == 0) {
178         free(cmd);
179         report("MultiByteToWideChar");
180         return -1;
181     }
182     free(cmd);
183
184     STARTUPINFO si;
185     ZeroMemory(&si, sizeof(si));
186     si.cb = sizeof(si);
187     si.dwFlags = STARTF_USESTDHANDLES;
188     si.hStdInput = (HANDLE)in->read;
189     si.hStdOutput = (HANDLE)out_pipe->write;
190     si.hStdError = GetStdHandle(STD_ERROR_HANDLE);
191
192     PROCESS_INFORMATION pi;
193     ZeroMemory(&pi, sizeof(pi));
194     if (!CreateProcessW(NULL, wcmd, NULL, NULL, TRUE, 0, NULL, NULL, &si, &
195         pi)) {
196         report("CreateProcessW");
197         return -1;
198     }
199
200     out->proc = (uintptr_t)pi.hProcess;
201     out->pid = (unsigned long)pi.dwProcessId;
202
203     int rc = 0;
204     if (!CloseHandle(pi.hThread)) {
205         report("CloseHandle(thread)");
206         rc = -1;
207     }
208     if (os_pipe_close_read(in) != 0)
209         rc = -1;
210     if (os_pipe_close_write(out_pipe) != 0)
211         rc = -1;
212     return rc;

```

```

213
214 unsigned long os_process_pid(const os_process *p)
215 {
216     return p->pid;
217 }
218
219 int os_process_wait(os_process *p, int *exit_code)
220 {
221     WaitForSingleObject((HANDLE)p->proc, INFINITE);
222     DWORD code = 0;
223     if (!GetExitCodeProcess((HANDLE)p->proc, &code)) {
224         report("GetExitCodeProcess");
225         return -1;
226     }
227     *exit_code = (int)code;
228     if (!CloseHandle((HANDLE)p->proc)) {
229         report("CloseHandle(process)");
230         return -1;
231     }
232     p->proc = OS_INVALID;
233     return 0;
234 }
235
236 int os_process_close(os_process *p)
237 {
238     if (p->proc != OS_INVALID) {
239         if (!CloseHandle((HANDLE)p->proc)) {
240             report("CloseHandle(process)");
241             return -1;
242         }
243         p->proc = OS_INVALID;
244     }
245     return 0;
246 }

```

Листинг 2: src/os_api_win.c

Реализация для Linux:

```

1  #include "os_api.h"
2
3  #include <errno.h>
4  #include <signal.h>
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <string.h>
8  #include <sys/types.h>
9  #include <sys/wait.h>
10 #include <unistd.h>
11
12 #define OS_INVALID ((uintptr_t)-1) // value for closed handle, UINTPTR_MAX
13
14 static void report(const char *msg)
15 {
16     fprintf(stderr, "[os_api:%s] errno %d: %s\n", msg, errno, strerror(errno)
        );

```

```

17 }
18
19 static void ignore_sigpipe(void)
20 {
21     static int done = 0;
22     if (!done) {
23         signal(SIGPIPE, SIG_IGN);
24         done = 1;
25     }
26 }
27
28 unsigned long os_self_pid(void)
29 {
30     return (unsigned long) getpid();
31 }
32
33 int os_pipe_create(os_pipe *p)
34 {
35     int fds[2];
36     if (pipe(fds) < 0)
37     {
38         report("pipe");
39         return -1;
40     }
41     p->read = (uintptr_t) fds[0];
42     p->write = (uintptr_t) fds[1];
43     return 0;
44 }
45
46 int os_pipe_close_read(os_pipe *p)
47 {
48     if (p->read != OS_INVALID)
49     {
50         if (close((int) p->read) < 0)
51         {
52             report("close(read)");
53             return -1;
54         }
55         p->read = OS_INVALID;
56     }
57     return 0;
58 }
59
60 int os_pipe_close_write(os_pipe *p)
61 {
62     if (p->write != OS_INVALID)
63     {
64         if (close((int) p->write) < 0)
65         {
66             report("close(write)");
67             return -1;
68         }
69         p->write = OS_INVALID;
70     }
71     return 0;
72 }
73
74 int os_pipe_read(os_pipe *p, void *buf, size_t size)

```

```

75 | {
76 |     ssize_t rd = read((int)p->read, buf, size);
77 |     if (rd < 0)
78 |     {
79 |         report("read(pipe)");
80 |         return -1;
81 |     }
82 |     return rd == (ssize_t)size ? 1 : 0;
83 | }
84 |
85 | int os_pipe_write(os_pipe *p, const void *buf, size_t size)
86 | {
87 |     ssize_t wr = write((int)p->write, buf, size);
88 |     if (wr != (ssize_t)size)
89 |     {
90 |         if (wr < 0)
91 |             report("write(pipe)");
92 |         else
93 |             fprintf(stderr, "[os_api] short write\n");
94 |         return -1;
95 |     }
96 |     return 0;
97 | }
98 |
99 | int os_process_spawn(os_process *out, const char *const argv[],
100 |                     os_pipe *in, os_pipe *out_pipe)
101 | {
102 |     ignore_sigpipe();
103 |
104 |     int in_r = (int)in->read;
105 |     int in_w = (int)in->write;
106 |     int out_r = (int)out_pipe->read;
107 |     int out_w = (int)out_pipe->write;
108 |
109 |     pid_t pid = fork();
110 |     if (pid < 0)
111 |     {
112 |         report("fork");
113 |         return -1;
114 |     }
115 |
116 |     if (pid == 0)
117 |     { // child process
118 |         if (dup2(in_r, STDIN_FILENO) < 0)
119 |         {
120 |             fprintf(stderr, "[os_api] child: dup2(in_r, STDIN) failed\n");
121 |             _exit(127);
122 |         }
123 |         if (dup2(out_w, STDOUT_FILENO) < 0)
124 |         {
125 |             fprintf(stderr, "[os_api] child: dup2(out_w, STDOUT) failed\n");
126 |             _exit(127);
127 |         }
128 |         if (os_pipe_close_read(in) != 0)
129 |         {
130 |             fprintf(stderr, "[os_api] child: close pipe end failed\n");
131 |             _exit(127);
132 |         }

```

```

133     if (os_pipe_close_write(in) != 0)
134     {
135         fprintf(stderr, "[os_api] child: close pipe end failed\n");
136         _exit(127);
137     }
138     if (os_pipe_close_read(out_pipe) != 0)
139     {
140         fprintf(stderr, "[os_api] child: close pipe end failed\n");
141         _exit(127);
142     }
143     if (os_pipe_close_write(out_pipe) != 0)
144     {
145         fprintf(stderr, "[os_api] child: close pipe end failed\n");
146         _exit(127);
147     }
148     const char *prog = argv[0];
149     if (strchr(prog, '/'))
150     {
151         execv(prog, (char *const *)argv);
152     }
153     else
154     {
155         char path[4096];
156         size_t len = strlen(prog);
157         if (len >= sizeof(path) - 2)
158         {
159             fprintf(stderr, "[os_api] child: program name too long\n");
160             _exit(127);
161         }
162         path[0] = '.';
163         path[1] = '/';
164         memcpy(path + 2, prog, len + 1);
165         execv(path, (char *const *)argv);
166     }
167     fprintf(stderr, "[os_api] child: execv(%s) failed\n", prog);
168     _exit(127);
169 }
170
171 // main process
172 out->proc = (uintptr_t)pid;
173 out->pid = (unsigned long)pid;
174
175 int rc = 0;
176 if (os_pipe_close_read(in) != 0)
177     rc = -1;
178 if (os_pipe_close_write(out_pipe) != 0)
179     rc = -1;
180 return rc;
181 }
182
183 unsigned long os_process_pid(const os_process *p)
184 {
185     return p->pid;
186 }
187
188 int os_process_wait(os_process *p, int *exit_code)
189 {
190     int status = 0;

```

```

191     if (waitpid((pid_t)p->proc, &status, 0) < 0)
192     {
193         report("waitpid");
194         return -1;
195     }
196     if (WIFEXITED(status))
197     {
198         *exit_code = WEXITSTATUS(status);
199     }
200     else if (WIFSIGNALED(status))
201     {
202         *exit_code = 128 + WTERMSIG(status);
203     }
204     else
205     {
206         *exit_code = -1;
207     }
208     p->proc = OS_INVALID;
209     return 0;
210 }
211
212 int os_process_close(os_process *p)
213 {
214     p->proc = OS_INVALID;
215     return 0;
216 }

```

Листинг 3: src/os_api_posix.c

Родительский процесс:

```

1  #include "os_api.h"
2
3  #include <float.h>
4  #include <stdio.h>
5  #include <string.h>
6
7  #define MAX_LINE 1024
8
9  static int read_line(char *buf, int max)
10 {
11     if (!fgets(buf, max, stdin))
12         return 0;
13     size_t n = strlen(buf);
14     while (n > 0 && (buf[n - 1] == '\n' || buf[n - 1] == '\r')) {
15         buf[--n] = '\0';
16     }
17     return 1;
18 }
19
20 static int is_finite_float(float x)
21 {
22     return x == x
23         && x <= FLT_MAX
24         && x >= -FLT_MAX;
25 }

```

```

26
27 int main(void)
28 {
29     char file_name[MAX_LINE];
30     int rc = 0;
31
32     printf("Enter file name for child: ");
33     fflush(stdout);
34     if (!read_line(file_name, sizeof(file_name)) || file_name[0] == '\0') {
35         fprintf(stderr, "No file name provided\n");
36         return 1;
37     }
38
39     os_pipe in; /* in: parent -> child */
40     os_pipe out; /* out: child -> parent */
41     if (os_pipe_create(&in) != 0) {
42         fprintf(stderr, "[parent] os_pipe_create(pipe1) failed\n");
43         return 1;
44     }
45     if (os_pipe_create(&out) != 0) {
46         fprintf(stderr, "[parent] os_pipe_create(pipe2) failed\n");
47         return 1;
48     }
49
50     const char *argv[] = {"child", file_name, NULL};
51
52     os_process proc;
53     if (os_process_spawn(&proc, argv, &in, &out) != 0) {
54         fprintf(stderr, "[parent] os_process_spawn failed\n");
55         return 1;
56     }
57
58     fprintf(stderr, "[parent pid=%lu] child started, pid=%lu\n",
59             os_self_pid(), os_process_pid(&proc));
60
61     printf("Enter any number of float numbers per line, space-separated.\n");
62     ;
63     printf("Empty line or EOF to finish.\n");
64
65     char line[MAX_LINE];
66     while (1) {
67         printf("> ");
68         fflush(stdout);
69         if (!read_line(line, sizeof(line)))
70             break;
71         if (line[0] == '\0')
72             break;
73
74         const char *p = line;
75         int count = 0;
76         int ok = 1;
77         while (ok) {
78             float x;
79             int n = 0;
80             if (sscanf(p, "%f%n", &x, &n) != 1)
81                 break;
82             if (n <= 0)
83                 break;

```

```

83
84     if (!is_finite_float(x)) {
85         fprintf(stderr, "[parent] input is not a finite float: skipped\
n");
86         p += n;
87         continue;
88     }
89
90     if (os_pipe_write(&in, &x, sizeof(x)) != 0) {
91         fprintf(stderr, "[parent] os_pipe_write failed\n");
92         rc = 1;
93         ok = 0;
94         break;
95     }
96
97     fprintf(stderr, "[parent pid=%lu] sent %.3f\n", os_self_pid(), x);
98
99     p += n;
100    count++;
101 }
102
103 if (!ok)
104     break;
105
106 if (count == 0) {
107     fprintf(stderr, "[parent] no numbers in line or bad values given,
skipped\n");
108 } else {
109     fprintf(stderr, "[parent pid=%lu] line done: %d number(s)\n",
os_self_pid(), count);
110 }
111 }
112 }
113
114 if (os_pipe_close_write(&in) != 0) {
115     fprintf(stderr, "[parent] os_pipe_close_write(in) failed\n");
116     rc = 1;
117 }
118 fprintf(stderr, "[parent] pipe1 closed, waiting for child's reply...\n")
;
119
120 float sum;
121 int r = os_pipe_read(&out, &sum, sizeof(sum));
122 if (r == 1) {
123     fprintf(stderr, "[parent pid=%lu] child reported sum = %.3f\n",
os_self_pid(), sum);
124 } else {
125     fprintf(stderr, "[parent] failed to read sum from child\n");
126     rc = 1;
127 }
128
129 if (os_pipe_close_read(&out) != 0) {
130     fprintf(stderr, "[parent] os_pipe_close_read(out) failed\n");
131     rc = 1;
132 }
133
134 int code = -1;
135 if (os_process_wait(&proc, &code) == 0) {
136     fprintf(stderr, "[parent] child exited with code %d\n", code);
137 } else {

```

```

138     fprintf(stderr, "[parent] os_process_wait failed\n");
139     rc = 1;
140 }
141
142 if (os_process_close(&proc) != 0) {
143     fprintf(stderr, "[parent] os_process_close failed\n");
144     rc = 1;
145 }
146 return rc;
147 }

```

Листинг 4: src/app/parent.c

Дочерний процесс:

```

1  #include "os_api.h"
2
3  #include <stdio.h>
4
5  int main(int argc, char *argv[])
6  {
7      if (argc < 2) {
8          fprintf(stderr, "usage: child <file_name>\n");
9          return 1;
10     }
11     const char *file_name = argv[1];
12
13     fprintf(stderr, "[child pid=%lu] started, file='%s'\n",
14             os_self_pid(), file_name);
15
16     FILE *fout = fopen(file_name, "w");
17     if (!fout) {
18         perror("fopen(output file)");
19         return 1;
20     }
21
22     float x;
23     float sum = 0.0f;
24     int count = 0;
25
26     while (fread(&x, sizeof(float), 1, stdin) == 1) {
27         sum += x;
28         count++;
29         fprintf(stderr, "[child pid=%lu] got %.3f, running sum = %.3f\n",
30                 os_self_pid(), x, sum);
31     }
32
33     fprintf(stderr, "[child pid=%lu] EOF on stdin, %d number(s) total\n",
34             os_self_pid(), count);
35
36     if (fprintf(fout, "Sum of %d numbers = %.6f\n", count, sum) < 0) {
37         perror("fprintf(output file)");
38         fclose(fout);
39         return 1;
40     }
41     fclose(fout);

```

```

42     fprintf(stderr, "[child pid=%lu] result written to '%s'\n",
43                 os_self_pid(), file_name);
44
45     if (fwrite(&sum, sizeof(sum), 1, stdout) != 1) {
46         perror("fwrite(stdout)");
47         return 1;
48     }
49     fflush(stdout);
50
51     fprintf(stderr, "[child pid=%lu] exiting\n", os_self_pid());
52     return 0;
53 }

```

Листинг 5: src/app/child.c

Файл сборки:

```

1  cmake_minimum_required(VERSION 3.15)
2
3  project(lab2 C)
4
5  set(CMAKE_C_STANDARD 11)
6  set(CMAKE_C_STANDARD_REQUIRED ON)
7  set(CMAKE_C_EXTENSIONS OFF)
8
9  set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
10
11 if(MSVC)
12     add_compile_options(/W4 /WX-)
13     add_compile_definitions(_CRT_SECURE_NO_WARNINGS)
14 else()
15     add_compile_options(-Wall -Wextra -Wpedantic)
16 endif()
17
18 add_library(os_api STATIC src/os_api.h)
19 target_include_directories(os_api PUBLIC src)
20
21 if(WIN32)
22     message(STATUS "Building for Windows (Win32 API)...")
23     target_sources(os_api PRIVATE src/os_api_win.c)
24     target_link_libraries(os_api PRIVATE kernel32)
25 elseif(UNIX AND NOT APPLE)
26     message(STATUS "Building for Linux (POSIX API)...")
27     target_sources(os_api PRIVATE src/os_api_posix.c)
28 elseif(APPLE)
29     message(FATAL_ERROR "macOS is not supported yet")
30 else()
31     message(FATAL_ERROR "Unsupported platform")
32 endif()
33
34 add_executable(parent src/app/parent.c)
35 target_link_libraries(parent PRIVATE os_api)
36
37 add_executable(child src/app/child.c)
38 target_link_libraries(child PRIVATE os_api)

```

Листинг 6: CMakeLists.txt

Ниже приведены ключевые системные вызовы родительского процесса под Linux (полная трасса — `strace -f -o strace.txt ./parent`, итог сохранён в `strace.txt`):

```
1 | execve("./parent", ["./parent"], ...) = 0
2 | pipe2([3, 4], 0) = 0 // pipe: in = [3, 4]
3 | pipe2([5, 6], 0) = 0 // pipe: out = [5, 6]
4 | rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, ...}, ...) = 0
5 | clone(child_stack=NULL, flags=...|SIGCHLD, ...) = 1303 // fork()
6 | close(3) = 0 // parent: in.read
7 | close(6) = 0 // parent: out.write
8 | dup2(3, 0) = 0 // child: stdin <- in.read
9 | dup2(6, 1) = 1 // child: stdout <- out.write
10 | close(4) = 0
11 | close(5) = 0
12 | execve("./child", ["child", "test_strace"], ...) = 0
13 | write(4, <1.0f = 0x3F800000>, 4) = 4 // parent: float 1.0 -> in.write
14 | read(3, <1.0f = 0x3F800000>, 4096) = 4 // child: reads 1.0
15 | write(4, <2.0f = 0x40000000>, 4) = 4 // parent: float 2.0 -> in.write
16 | read(3, <2.0f = 0x40000000>, 4096) = 4 // child: reads 2.0
17 | write(4, <3.0f = 0x40400000>, 4) = 4 // parent: float 3.0 -> in.write
18 | read(3, <3.0f = 0x40400000>, 4096) = 4 // child: reads 3.0
19 | write(4, <0.0f = 0x00000000>, 4) = 4 // parent: float 0.0 -> in.write
20 | read(3, <0.0f = 0x00000000>, 4096) = 4 // child: reads 0.0
21 | close(4) = 0 // parent: EOF for child
22 | read(3, "", 4096) = 0 // child: sees EOF
23 | write(1, <6.0f = 0x40C00000>, 4) = 4 // child: sum 6.0 -> stdout
24 | read(5, <6.0f = 0x40C00000>, 4) = 4 // parent: reads sum
25 | wait4(1303, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 1303
26 | exit_group(0) = ?
```

Листинг 7: `strace` (fragment)

Трассировка подтверждает описанный ход работы: создание двух каналов (`pipe2`), подавление `SIGPIPE` (`rt_sigaction`), запуск дочернего процесса (`clone`), подключение каналов к стандартному вводу/выводу (`dup2`) с последующим `execve("./child")`, передачу чисел и итоговой суммы (4 байта в младшей части типа `float`), корректную передачу EOF путём закрытия записывающего конца (`close(4) → read = 0`) и ожидание завершения процесса (`wait4`) с кодом выхода 0.